The MORE Architecture – A Structured and Maintainable Alternative to MVC for Web Applications

Kamel Zerki

July, 2025. *Algiers, Algeria*
securyh@gmail.com

Abstract

The Model-View-Controller (MVC) paradigm has long dominated web application architecture. However, modern web development increasingly demands simplicity, maintainability, and performance over abstraction-heavy designs. This paper proposes MORE (Models, Operations, Renders, Events) — a minimalistic and structured architecture designed to replace MVC in lightweight and mid-sized web applications. MORE cleanly separates data fetching, data mutations, rendering, and routing, addressing many of MVC's known limitations in PHP-based ecosystems. This whitepaper discusses the conceptual motivations, architectural design, comparative analysis, and suitability of MORE for general-purpose applications, especially within educational and professional development contexts.

1. Introduction

In the evolution of web application architectures, MVC has been widely accepted as a default organizational pattern. Yet, its increasing complexity and the steep learning curve associated with full-stack implementations such as Laravel and Symfony have prompted developers to seek lighter alternatives [1].

This whitepaper introduces MORE, an architecture developed from first principles, aiming to reduce overhead and enhance code clarity without sacrificing capability. Unlike MVC, which centralizes control logic and often encourages poor separation of responsibilities, MORE divides application concerns into four distinct layers: Models, Operations, Renders, and Events.

2. Problem Statement

Despite MVC's pervasiveness, it presents structural challenges:

- Controllers frequently become bloated with both business logic and routing responsibilities [2].
- View templates are often tightly coupled to controller logic or templating engines [3].
- Data mutations (create/update/delete operations) are often indistinguishable from pure fetch logic within models, reducing clarity.

This fragmentation increases the difficulty of onboarding new developers and slows down feature delivery in lean development teams.
Additionally, the increasing complexity of mainstream frameworks contributes to what many developers describe as *framework fatigue* — a phenomenon where the learning curve, configuration overhead, and tightly-coupled design patterns of large frameworks lead to decreased productivity and burnout, particularly in small teams and solo projects [4]. MORE addresses this challenge by offering a pragmatic, minimalistic architecture tailored for rapid onboarding and sustainable long-term development.

3. The MORE Architecture

MORE redefines the division of concerns across four distinct components:

3.1. Models

Models in MORE are read-only data access layers. Their sole responsibility is to query databases or data sources and return structured results. By enforcing this unidirectionality, MORE eliminates side effects from read operations.

3.2. Operations

Operations are *explicit command classes* responsible for writing to the database, modifying files, or performing any I/O-heavy process. This strict separation helps developers quickly trace state-changing logic and introduces predictable behavior [5].

3.3. Renders

Render classes inherit from abstract view controllers. They are responsible for:

- Selecting the correct HTML templates.
- Injecting the required data from models.
- Handling input views and partials.

MORE promotes rendering logic as a standalone concern to reduce coupling with business logic and support reuse across themes and UIs.

3.4. Events

Events act as the routing system — lightweight dispatchers that interpret incoming HTTP requests (or sockets, in real-time systems) and map them to the corresponding render or operation. This event-based design simplifies route definitions and supports programmatic extensibility.

4. Conceptual Benefits

The MORE architecture achieves the following benefits:

- **Predictability:** Every concern has a clearly defined role, minimizing confusion in logic flow.
- **Maintainability:** Codebases built with MORE are easier to audit, refactor, and extend.
- **Security:** By isolating mutations and I/O in Operations, developers can apply stricter validations and audit trails.
- **Ease of Learning:** Developers new to PHP or coming from procedural backgrounds can adopt MORE quickly compared to controller-heavy MVC systems [6].

5. Comparison With MVC

Table 1 Comparison between MORE and MVC.

Feature	*MVC (Laravel)*	*MORE Architecture*
Control Layer Complexity	Centralized & often bloated	Distributed across Events & Renders
Data Mutation Separation	Poor	Strictly isolated in Operations
View Layer Coupling	High (Blade templates & Controllers)	Low (Pure HTML + Data Injection)
Plugin Overhead	Medium-High	Currently Low
Learning Curve	Medium–High	Low–Medium
Ideal Use Cases	Enterprise/Monoliths	CMS, Panels, API Backends, Custom Apps

6. Design Patterns & Practices.

MORE encourages:

- No shared logic between read and write operations.
- Zero logic in templates.
- Full HTML flexibility (no vendor lock-in templating engines).
- Loose coupling between rendering and data storage.
- Optional CLI tooling for database setup and packaging.

7. Security Considerations

Security is a first-class concern in MORE:
By cleanly isolating data input, output, and execution paths, the architecture mitigates typical web application threats such as SQL injection, CSRF, and XSS. Strict naming conventions and centralized request parsing further improve input validation. The system is also naturally supportive of role-based access control, as operations can be easily isolated and verified.

This structural clarity minimizes injection points and reduces the risk of unintended data exposure [7].

8. Educational Value

For learners, MORE serves as a gentle transition from procedural PHP to structured application development. Its strict discipline on logic separation mimics academic concepts such as pure functions, single responsibility, and encapsulation without requiring formal OOP mastery.

9. Potential Extensions

The MORE architecture is designed to remain adaptable to the evolving needs of web system design. Its emphasis on strict separation of concerns and clarity in code flow makes it suitable for integration into educational curricula, modular frameworks, and domain-specific platforms. Potential avenues for future exploration include graphical development interfaces, structural design assistants, and formal specification layers for routing and access control.

A complete implementation of the MORE architecture has been realized in a functional software framework (Morphine) [8], which validates its applicability in production-grade development environments. The source code and architectural walkthrough are publicly available and serve as a practical complement to the theoretical model presented herein.

10. Conclusion

The MORE architecture is a practical and elegant alternative to MVC, especially suitable for developers seeking to build flexible, lightweight, and secure PHP applications without the overhead of traditional frameworks. Its clean division of responsibilities, emphasis on maintainability, and support for real-world use cases make it a strong candidate for both professional and academic environments.

Competing Interests

The author has declared that no competing interests exist.

Authors' contribution

Kamel Zerki conceived and designed the MORE architecture, developed its core concepts, and authored this whitepaper. The author independently implemented the architecture in practice, analyzed its advantages compared to conventional patterns such as MVC, and provided real-world validation through the Morphine framework. All work was carried out independently by a single author. The author read and approved the final manuscript.

References

[1] Fowler, M., *Patterns of Enterprise Application Architecture*. Addison-Wesley., 2003.

[2] Otwell, T., Laravel Documentation – Controllers. https://laravel.com/docs, 2011.

[3] Suryavanshi, D., "*MVC Anti-Patterns*," Journal of Web Architecture Studies, 2018.

[4] Vepsäläinen, J., Hellas, A., & Vuorimaa, P. "*The Rise of Disappearing Frameworks in Web Development*". arXiv. 2023

[5] Martin, R. C., "*Clean Code: A Handbook of Agile Software Craftsmanship*" Prentice Hall, 2009.

[6] Mijatović, J., "Why MVC May Be Overkill in Small PHP Apps". Medium Article, 2020.

[7] OWASP Foundation, OWASP Top Ten Security Risks Available at: https://owasp.org/www-project-top-ten/ .

[8] Zerki K., "*Applied Implementation of the MORE Architecture in the Morphine Framework*," Morphine GitHub Wiki, 2025. Available at: https://github.com/0x31337/morphine/wiki/Applied-Implementation-of-the-MORE-Architecture-in-the-Morphine-Framework